

EECS2021

YORK
UNIVERSITÉ
UNIVERSITY 

Fall 2025

Instructor:

Amirali Amirsoleimani

Associate Professor

York University

Instructor Short Biography

■ Instructor Bio: Amirali Amirsoleimani

- Associate professor EECS York University, PhD, PEng
- Director of **Lab for Computing Research and Innovation (LCRAIN)**
- Address: Room 2020, Lassonde Building, York University
- Biography Detail: <https://lassonde.yorku.ca/users/amirsol>



- **Openings:** Students interested in co-hardware/software design can join the research group.
Contact: amirsol@yorku.ca

Course Detail and Personnel

■ **Instructor:** Amirali Amirsoleimani amirsol@yorku.ca

Office hours: Tuesdays (10:00 AM EST to 11:00 AM EST)

Room 2020 Lassonde Building

■ **TAs:** TBA

Office hours: Can be find in eClass

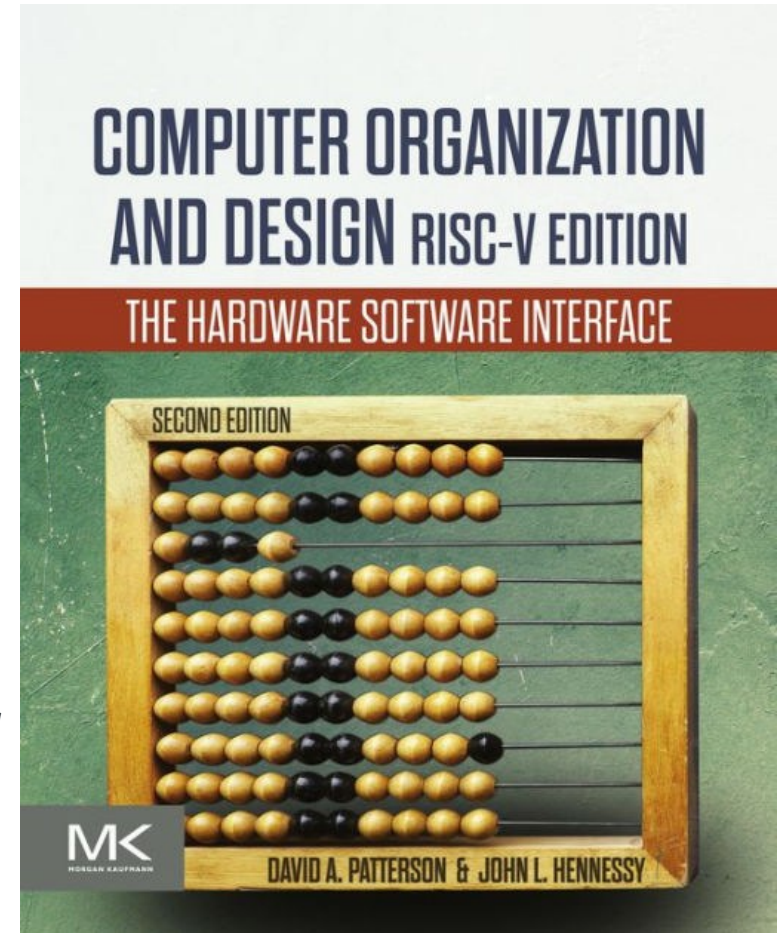
Text Book

■ Text book for EECS2021:

Computer Organization and Design
RISC-V Edition: The Hardware
Software Interface

by David Patterson, John L. Hennessy

“Focuses on 32-bit ISA rather than 64-bit RISC-V because 32-bit RISC-V ISA is simpler to explain, and 32-bit address computers are still best for applications like”



eClass for EECS 2021

LE/EECS 2021 F - Computer Organization (Fall 2024-2025)

+ Add a block

Waiting List



IF YOU ARE IN WAITING LIST OR PLAN TO ENROLL IN EECS2021?

Students who are not officially enrolled in the course (and plan to enroll) are expected to attend the lectures and labs from the first week of the term and complete all required works within the deadlines. Please contact the course director for details on how to obtain *temporary* access to the course materials while your enrolment status is being decided. Otherwise, if you miss any assessment deadline, you will receive 0 for that assessment. For official enrollment to the course, you must contact the UG office by emailing eeecs-ug@yorku.ca, or use the EECS UG Office queue on QLess to talk to an advisor. Please note that engineering students must enroll themselves only in section E.

▼ General

Collapse all 

Here, you can download the detailed [course syllabus](#).

EECS2021:

Introduction to computer organization and instruction set architecture, covering assembly language, machine language and encoding, addressing modes, single/multicycle datapaths (including functional units and controls), pipelining, memory segments and memory hierarchy.

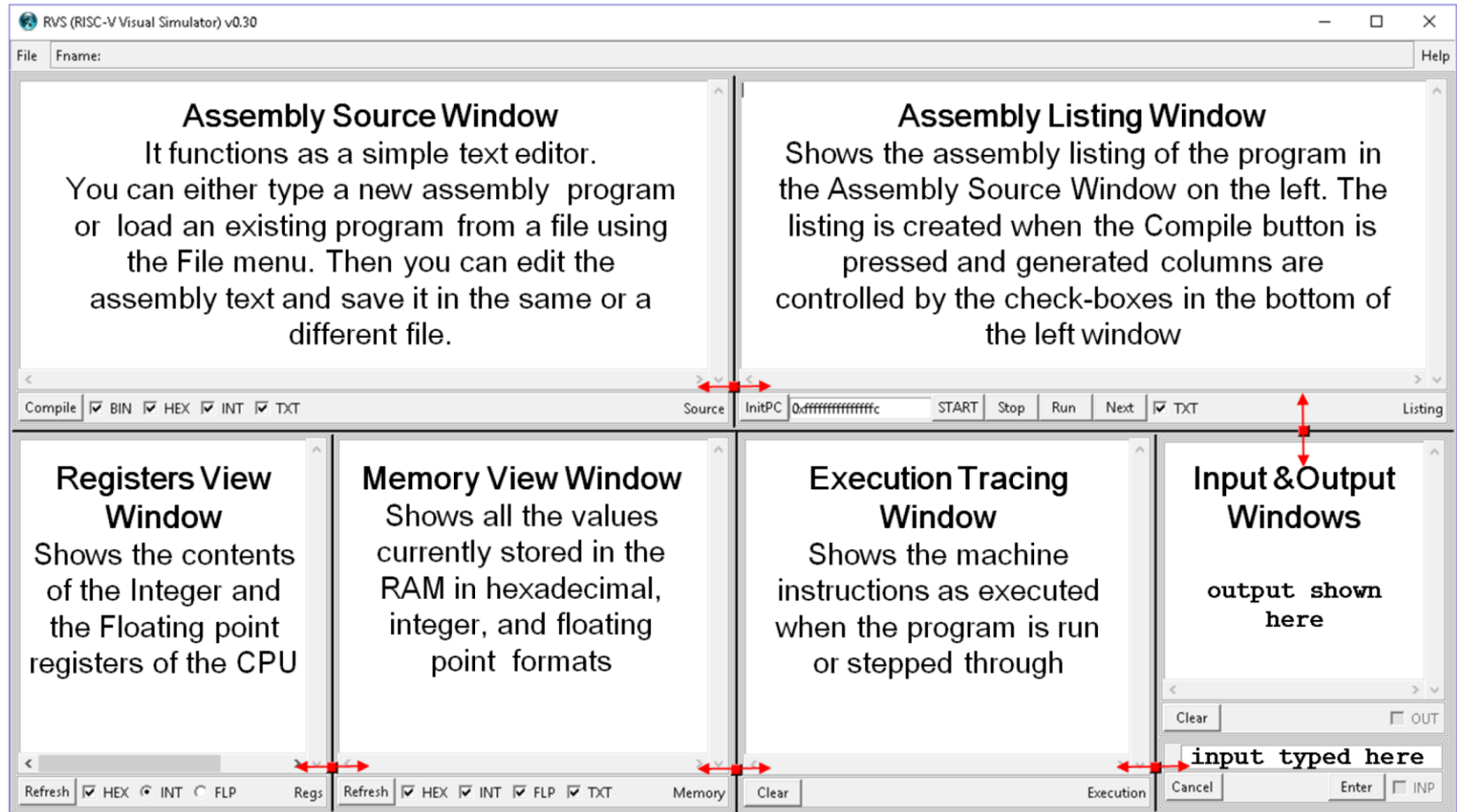
Prerequisites: cumulative GPA of 4.50 or better over all major EECS courses (without second digit "5"); LE/EECS 1021 3.00 or LE/EECS 1022 3.00 or LE/EECS 1720 3.00 or LE/EECS 1030 3.00. Previously offered as: LE/CSE 2021 4.00, SC/CSE 2021 4.00.

Prerequisites

- EECS 1022 (If not it may be also fine)
- Having the basic programming knowledge
- Labs (@LAS 1006): We will use RISC-V in-house YorkU simulator for the labs.

RISC-V Simulator

- Here is the interface for RISC-V simulator



RISC-V Simulator (Cont.)

RVS (RISC-V Visual Simulator) v0.46

File Fname: Help

```
;calculates 4!=4*3*2*1=24
    addi    x5, x0, 4
    addi    x6, x0, 1
loop:  mul   x6, x6, x5
    addi    x5, x5, -1
    ebreak  x0, x0, 0
    blt     x0, x5, loop
```

ASSEMBLY LISTING

ADDRESS	BIN/HEX CODE	TEXT SOURCE
0x0000000000000000	I 0000000000000000 00000 000 00101 0010011	addi x5, x0, 4
0x0000000000000004	I 0000000000000001 00000 000 00110 0010011	addi x6, x0, 1
0x0000000000000008	R 00000001 00101 00110 000 00110 0110011	loop: mul x6, x6, x5
0x000000000000000c	I 111111111111 00101 000 00101 0010011	addi x5, x5, -1
0x0000000000000010	I 00000000000000 00000 000 00000 1110011	ebreak x0, x0, 0
0x0000000000000014	SB 1111111 00101 00000 100 11011 1100011	blt x0, x5, loop

SYMBOL TABLE

```
0x0000000000000000 START
0x0000000000000008 loop
```

Compile ☒ BIN ☐ HEX ☐ INT ☒ TXT Source InitPC 0x0000000000000018 START Stop Run Next ☒ TXT Listing

INT Regs	ADDRESS	HEXADECIMAL
x0 zero	0x0000000000000000	0
x1 ra	0x0000000000000000	0
x2 sp	0x0000000000000000	0
x3 gp	0x0000000000000000	0
x4 tp	0x0000000000000000	0
x5 t0	0x0000000000000000	0
x6 t1	0x0000000000000018	24
x7 t2	0x0000000000000000	0
x8 s0	0x0000000000000000	0
x9 s1	0x0000000000000000	0
x10 a0	0x0000000000000000	0
x11 a1	0x0000000000000000	0
x12 a2	0x0000000000000000	0
x13 a3	0x0000000000000000	0
x14 a4	0x0000000000000000	0
x15 a5	0x0000000000000000	0
x16 a6	0x0000000000000000	0
x17 a7	0x0000000000000000	0
x18 s2	0x0000000000000000	0
x19 s3	0x0000000000000000	0
x20 s4	0x0000000000000000	0
x21 s5	0x0000000000000000	0
x22 s6	0x0000000000000000	0
x23 s7	0x0000000000000000	0
x24 s8	0x0000000000000000	0
x25 s9	0x0000000000000000	0
x26 s10	0x0000000000000000	0

START

```
0x0000000000000000 addi x5 x0 0x004 addi x5, x0, 4
0x0000000000000004 addi x6 x0 0x001 addi x6, x0, 1
0x0000000000000008 mul x6 x6 x5 loop: mul x6, x6, x5
0x000000000000000c addi x5 x5 0xffff addi x5, x5, -1
0x0000000000000010 ebreak x0 x0 0x000 ebreak x0, x0, 0
ebreak
RUN
0x0000000000000014 blt x0 x5 0xffa blt x0, x5, loop
0x0000000000000008 mul x6 x6 x5 loop: mul x6, x6, x5
0x000000000000000c addi x5 x5 0xffff addi x5, x5, -1
0x0000000000000010 ebreak x0 x0 0x000 ebreak x0, x0, 0
ebreak
RUN
0x0000000000000014 blt x0 x5 0xffa blt x0, x5, loop
0x0000000000000008 mul x6 x6 x5 loop: mul x6, x6, x5
0x000000000000000c addi x5 x5 0xffff addi x5, x5, -1
0x0000000000000010 ebreak x0 x0 0x000 ebreak x0, x0, 0
ebreak
RUN
0x0000000000000014 blt x0 x5 0xffa blt x0, x5, loop
ERROR: 0x0000000000000018: NO INSTRUCTION
```

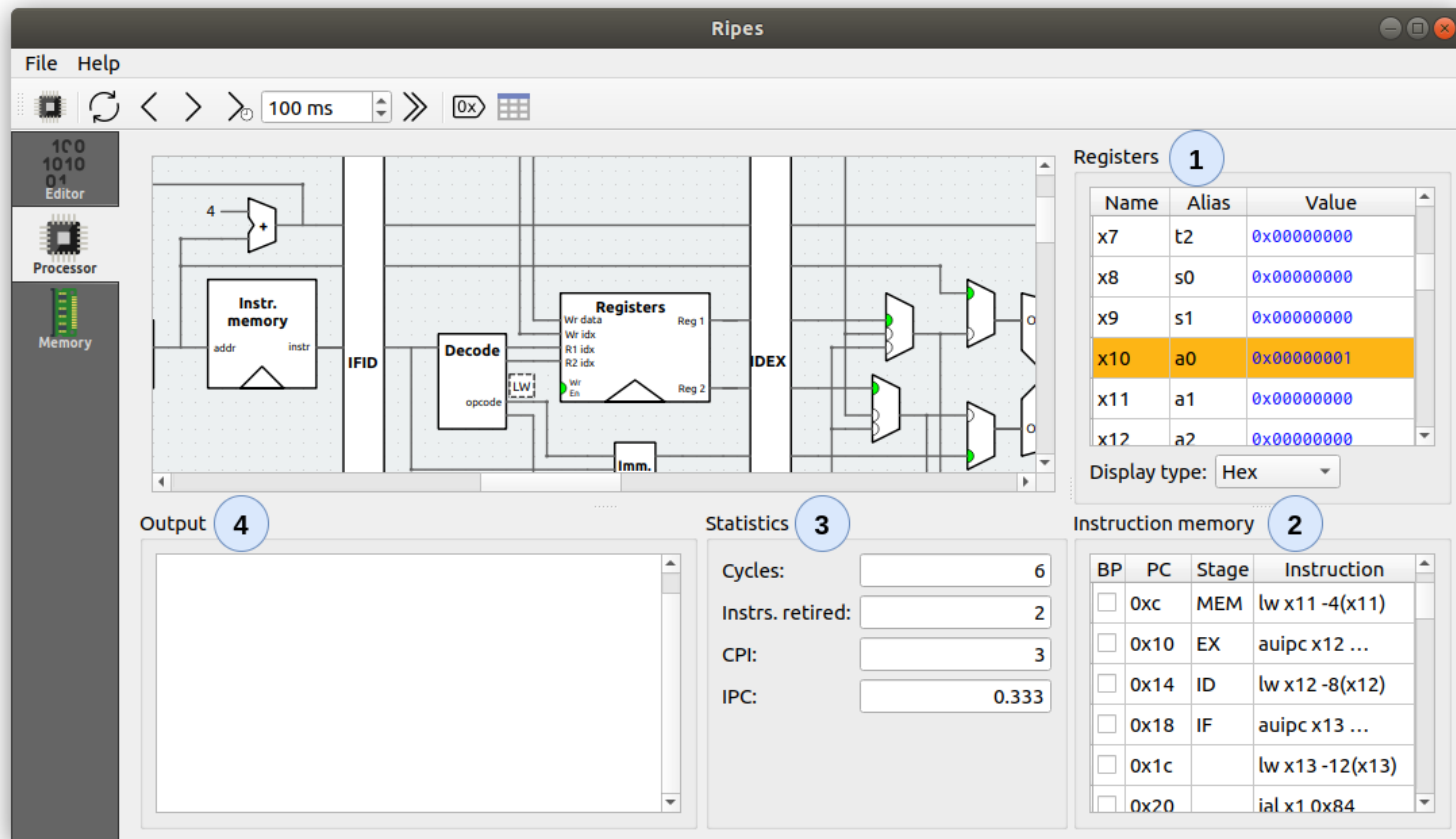
Refresh ☒ HEX ☒ INT ☒ FLP Regs Refresh ☒ HEX ☒ INT ☒ FLP ☒ TEXT Mem Clear Exec

- You should have the kit.
- YorkU Library will have it available.
- Each person should have one.



Ripe Simulator

- For pipelining or Cache memory labs



Prerequisites

- EECS 1022 (If not it may be also fine)
- Having the basic programming knowledge
- Labs (@LAS 1006): We will use RISC-V in-house YorkU simulator for the labs.

Grade Composition

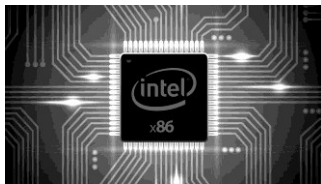
- Labs: **40%** (8 Labs each with Lab test)
 - Midterm: **20%**
 - Final: **40%**
-
- Labs are 3 hours (1 Lab Tests 16%)
 - Lab books will be shared earlier than the labs
 - In the lab you will do lab quiz and submit files.

Training Lab

- **Lab T** is a training lab designed to help you familiarize yourself with the employed computing environment, software, and related procedures.
- You can use both remote connection or installing RVS and ICARUS Verilog on your own PC.
- TAs will help during the lab and available to answer your questions.

EECS2021 Course

- Reduced instruction set computing (RISC) vs. Complex instruction set computing (CISC)
- We focus on RISC-V ISA and will discuss its architecture.
- RISC-V is the first open-source ISA and widely used in modern computing environments such as cloud computing and mobile devices.



Intel - X86



RISC-V



ARM

MIPS

MIPS

What You Will Learn

- How programs are translated into the machine language
 - And how the hardware executes them
- The hardware/software interface
- What determines program performance
 - And how it can be improved
- How hardware designers improve performance
- What is parallel processing

EECS2021 (Book)

- **Chapter 1** – Computer Abstractions and Technologies
- **Chapter 2** – Instructions, Language of Computer
- **Chapter 3** – Arithmetic for Computer
- **Appendix A** – The Basics of Logic Design
- **Chapter 4** – Processor (Pipelining)
- **Chapter 5** – Exploiting Memory Hierachy

Chapter 1

Computer Abstractions and Technology

The Computer Revolution

- Progress in computer technology
 - Underpinned by domain-specific accelerators
- Makes novel applications feasible
 - Computers in automobiles
 - Cell phones
 - Human genome project
 - World Wide Web
 - Search Engines
- Computers are pervasive

Classes of Computers

- Personal computers
 - General purpose, variety of software
 - Subject to cost/performance tradeoff
- Server computers
 - Network based
 - High capacity, performance, reliability
 - Range from small servers to building sized

Classes of Computers

- Supercomputers
 - Type of server
 - High-end scientific and engineering calculations
 - Highest capability but represent a small fraction of the overall computer market
- Embedded computers (Edge Computing)
 - Hidden as components of systems
 - Stringent power/performance/cost constraints

Tesla Supercomputer



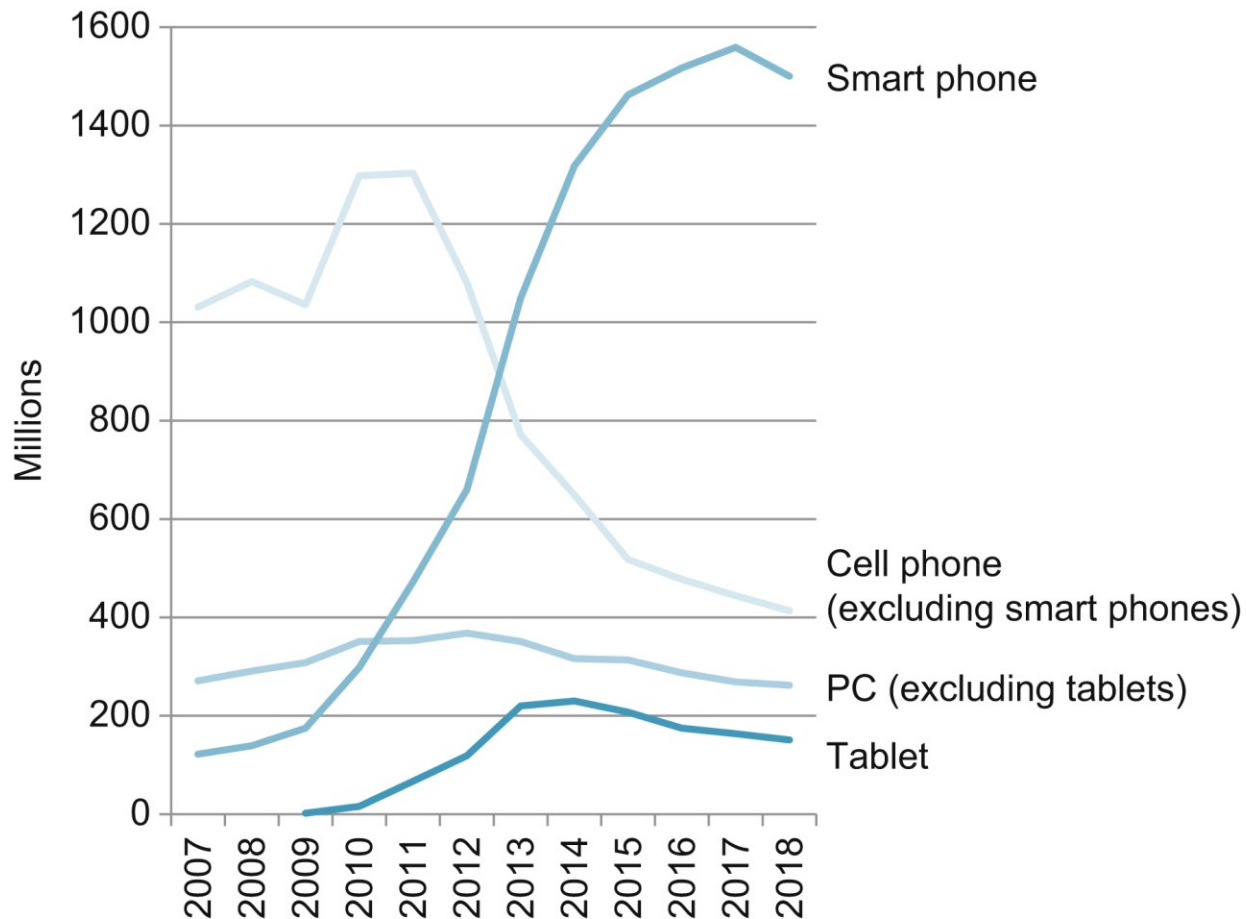
- Tesla unveils its new supercomputer (5th most powerful in the world) to train self-driving AI. 720 nodes of 8x A100 80GB. (5760 GPUs total)
- 1.8 EFLOPS (720 nodes * 312 TFLOPS-FP16-A100 * 8 gpu/nodes)

Size Notations

Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%
ronnabyte	RB	10^{27}	robibyte	RiB	2^{90}	24%
queccabyte	QB	10^{30}	quebibyte	QiB	2^{100}	27%

The 2^x vs. 10^y bytes ambiguity was resolved by adding a binary notation for all the common size terms.

The PostPC Era



The PostPC Era

- Personal Mobile Device (PMD)
 - Battery operated
 - Connects to the Internet
 - Hundreds of dollars
 - Smart phones, tablets, electronic glasses
- Cloud computing
 - Warehouse Scale Computers (WSC) (Amazon and Google)
 - Software as a Service (SaaS) to PMDs
 - Portion of software run on a PMD and a portion run in the Cloud

Understanding Performance

- Algorithm
 - Determines number of operations executed
- Programming language, compiler, architecture
 - Determine number of machine instructions executed per operation
- Processor and memory system
 - Determine how fast instructions are executed
- I/O system (including OS)
 - Determines how fast I/O operations are executed

Seven Great Ideas

- Use ***abstraction*** to simplify design
- Make the ***common case fast***
- Performance *via* ***parallelism***
- Performance *via* ***pipelining***
- Performance *via* ***prediction***
- ***Hierarchy*** of memories
- ***Dependability*** *via* redundancy



ABSTRACTION



COMMON CASE FAST



PARALLELISM



PIPELINING



PREDICTION

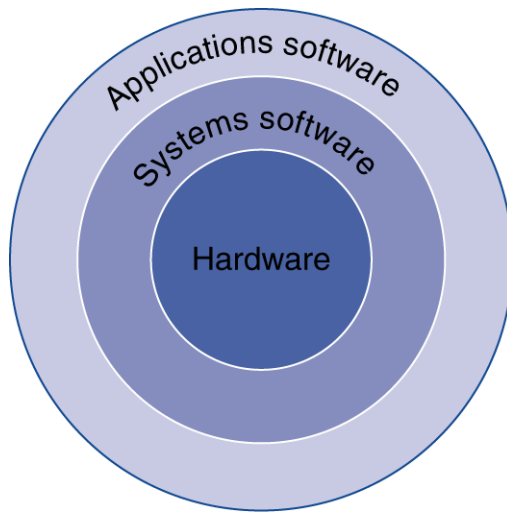


HIERARCHY



DEPENDABILITY

Below Your Program



- Application software
 - Written in high-level language
- System software
 - Compiler: translates HLL code to machine code
 - Operating System: service code
 - Handling input/output
 - Managing memory and storage
 - Scheduling tasks & sharing resources
- Hardware
 - Processor, memory, I/O controllers

Levels of Program Code

- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

```
swap:
  slli x6, x11, 3
  add x6, x10, x6
  ld x5, 0(x6)
  ld x7, 8(x6)
  sd x7, 0(x6)
  sd x5, 8(x6)
  jalr x0, 0(x1)
```

Assembler

Binary machine
language
program
(for RISC-V)

```
00000000001101011001001100010011
00000000011001010000001100110011
00000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
00000000000000001000000011001111
```

Programming Languages

- In HLL: $H = A + B + C$

In Assembly: `add X1, X10, X11`

`add X1, X1, X12`

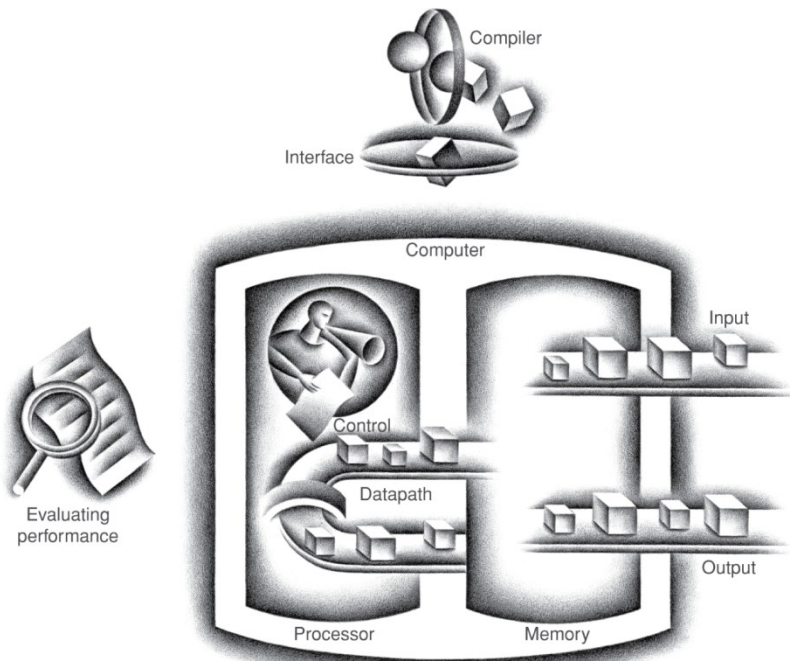
- Similar to natural language
- More productive with fewer lines:

Domain specific languages like PyTorch for ML, FORTRAN scientific computing, Cobol business data processing

- Final advantage: independent from the hardware

Components of a Computer

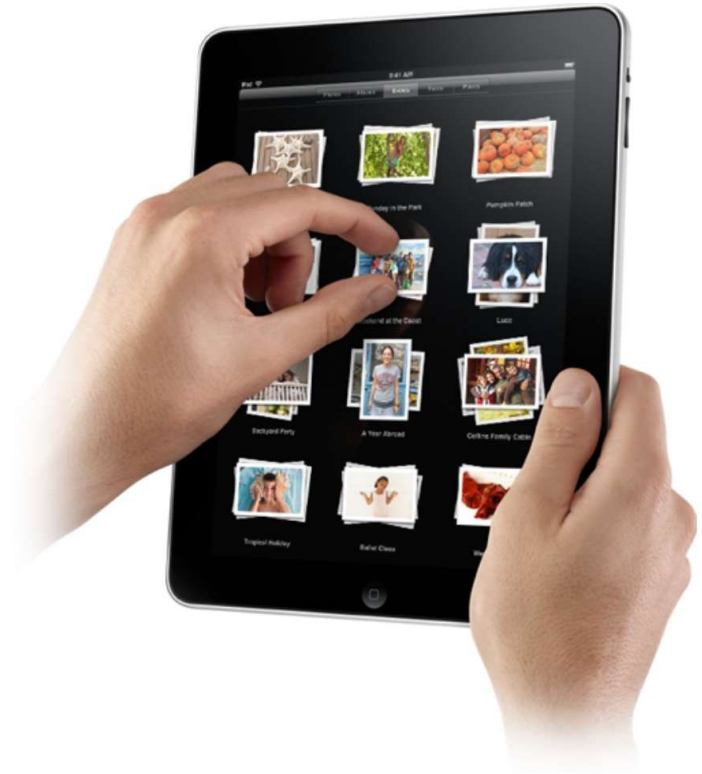
The BIG Picture



- Same components for all kinds of computer
 - Desktop, server, embedded
- Input/output includes
 - User-interface devices
 - Display, keyboard, mouse
 - Storage devices
 - Hard disk, CD/DVD, flash
 - Network adapters
 - For communicating with other computers

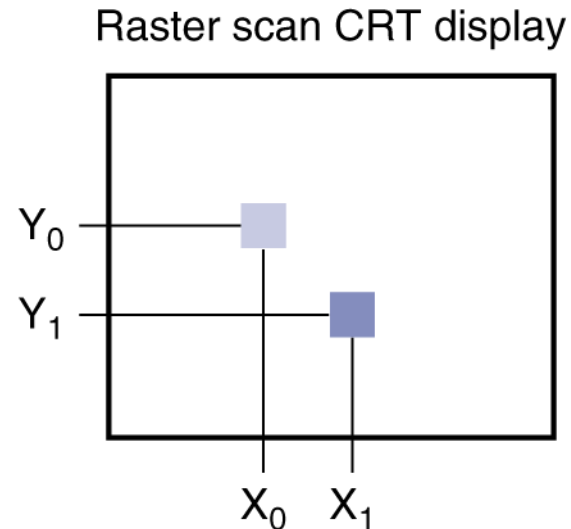
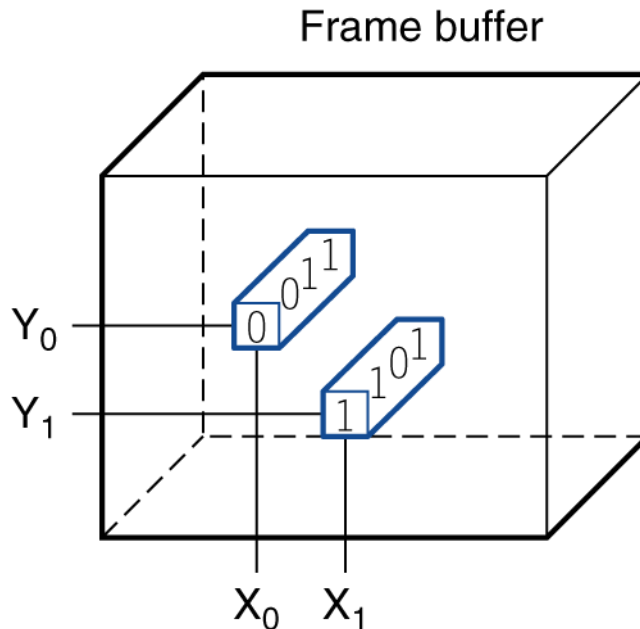
Touchscreen

- PostPC device
- Supersedes keyboard and mouse
- Resistive and Capacitive types
 - Most tablets, smart phones use capacitive
 - Capacitive allows multiple touches simultaneously

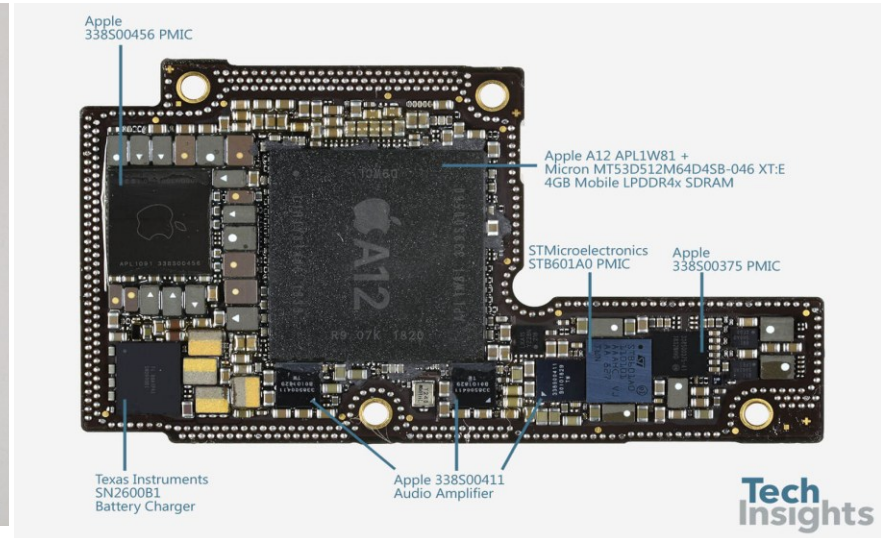
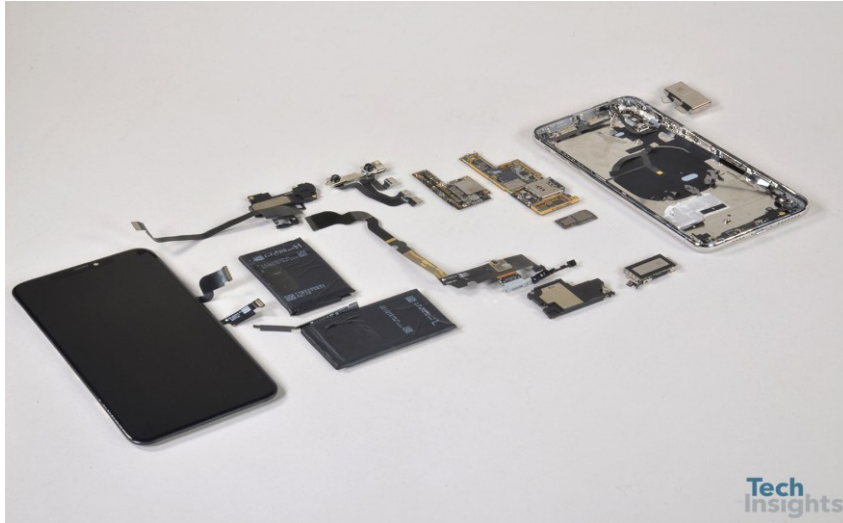


Through the Looking Glass

- LCD screen: picture elements (pixels)
 - Mirrors content of frame buffer memory



Opening the Box

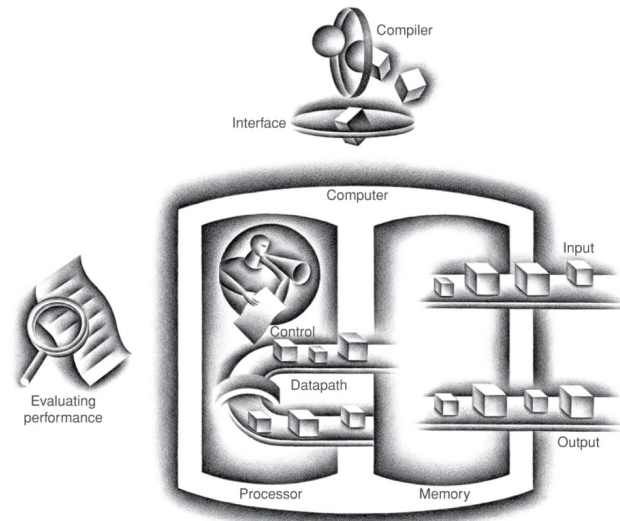


The logic board of Apple iPhone XS Max.

The large integrated circuit in the middle is the Apple A12 chip, which contains two large and four small ARM processor cores that run at 2.5 GHz, as well as 2 GiB of main memory inside the package.

Inside the Processor (CPU)

- Datapath: performs operations on data
- Control: sequences datapath, memory, ...
- Cache memory
 - Small fast SRAM memory for immediate access to data



Inside the Processor

■ A12 processor

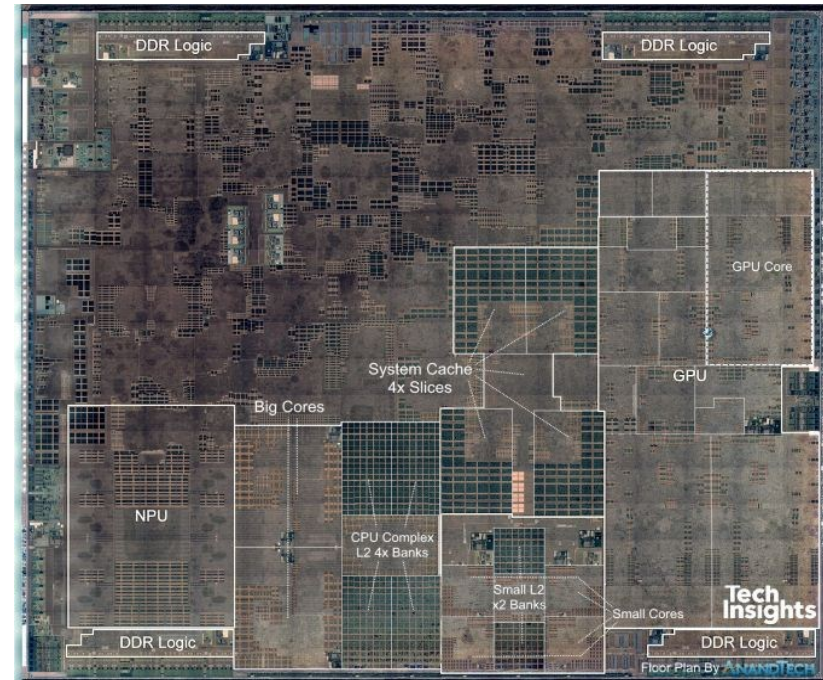
It has two identical ARM processors or cores.

Four small cores, a graphics processing unit (GPU) and a domain-specific accelerator for neural networks called the NPU.

In the middle are second-level cache memory (L2) banks for the big and small cores.

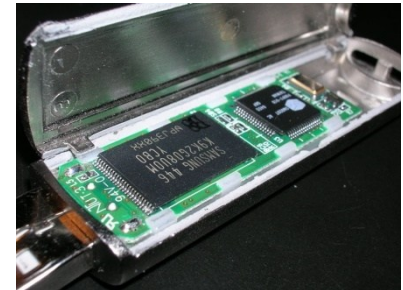
At the top and bottom of the chip are interfaces to the main memory (DDR DRAM).

8.4 mm x 9.91 mm



A Safe Place for Data

- Volatile main memory
 - Loses instructions and data when power off
- Non-volatile secondary memory
 - Magnetic disk
 - Flash memory
 - Optical disk (CDROM, DVD)



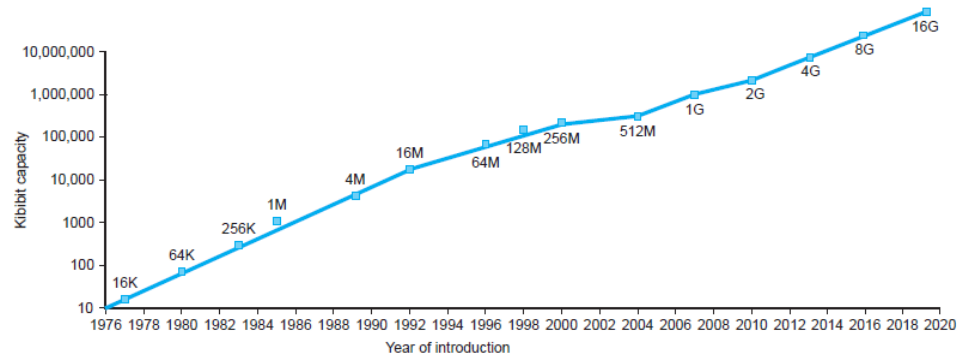
Abstractions

The BIG Picture

- Abstraction helps us deal with complexity
 - Hide lower-level detail
- Instruction set architecture (ISA)
 - The hardware/software interface
- Application binary interface
 - The ISA plus system software interface
- Implementation
 - The details underlying and interface

Technology Trends

- Electronics technology continues to evolve
 - Increased capacity and performance
 - Reduced cost

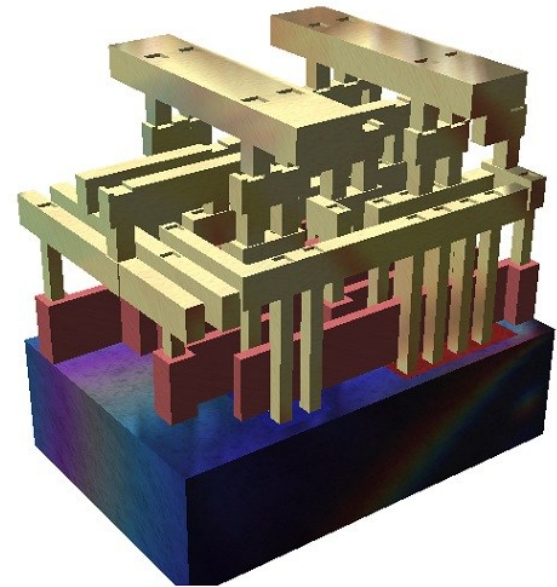
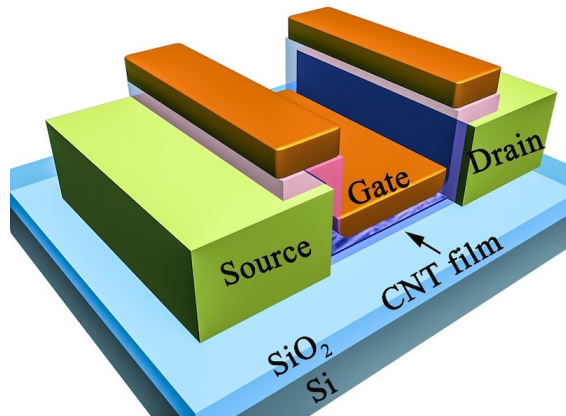


DRAM capacity

Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2013	Ultra large scale IC	250,000,000,000

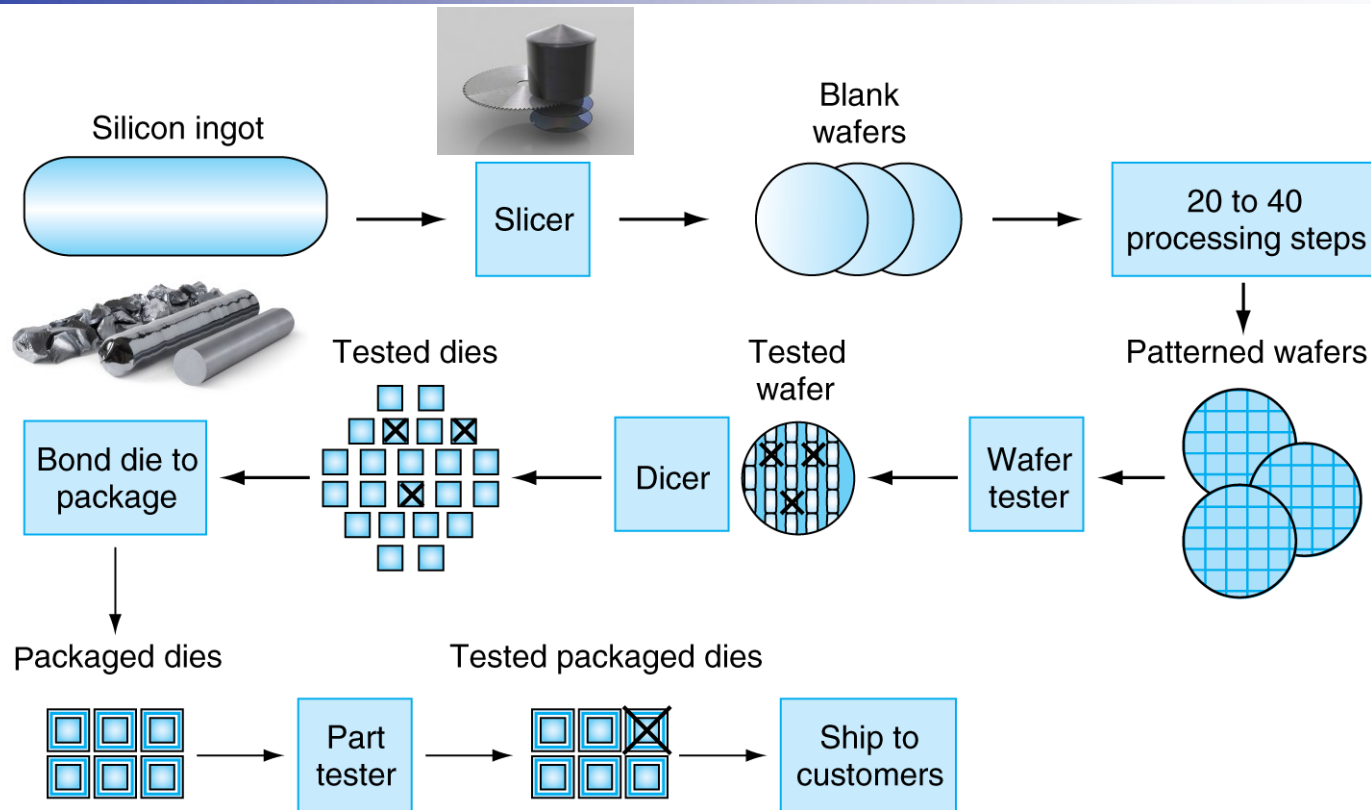
Semiconductor Technology

- Silicon: semiconductor
- Add materials to transform properties:
 - Conductors
 - Insulators
 - Switch



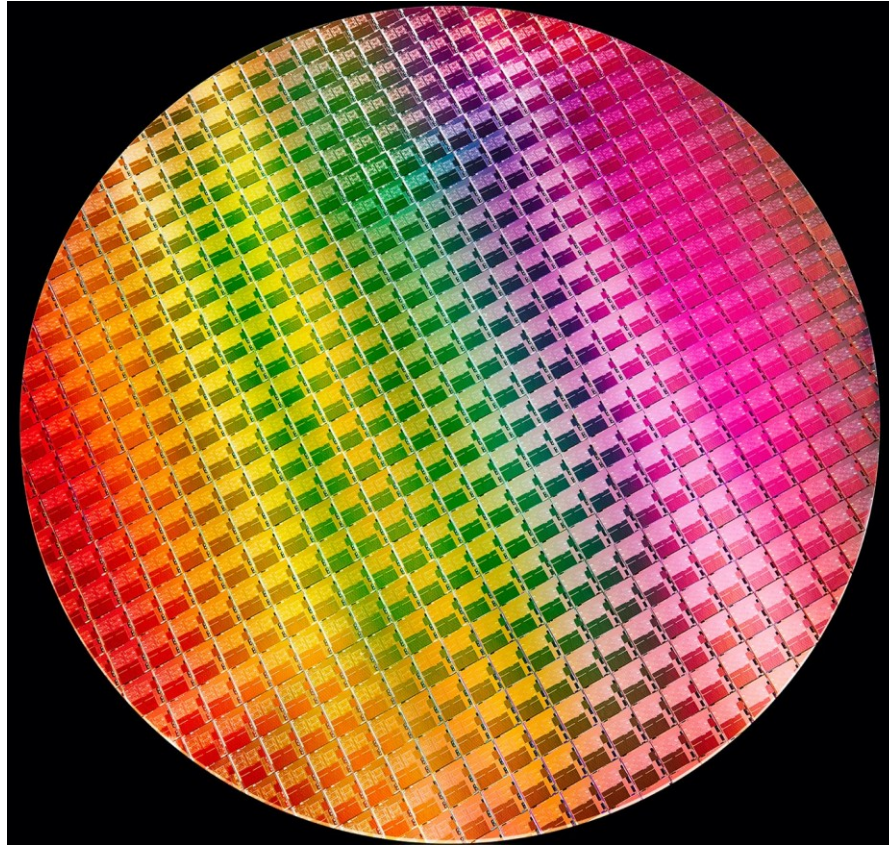
[Carbon nanotube films make gigahertz integrated circuits - News in field - NEWS - Chinese Academy of Sciences, Chengdu Organic Chemistry Co., Ltd. \(timesnano.com\)](#)

Manufacturing ICs



- Yield: proportion of working dies per wafer

Intel® Core 10th Gen



- 300mm wafer, 506 chips, 10nm technology
- Each chip is 11.4 x 10.7 mm

Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

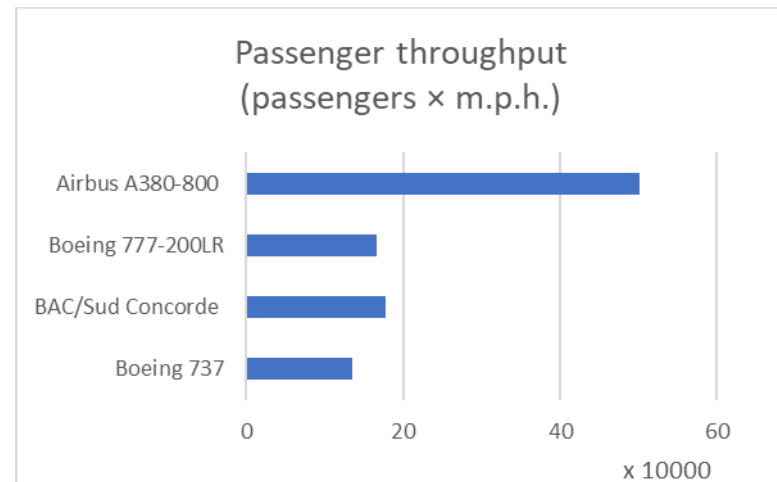
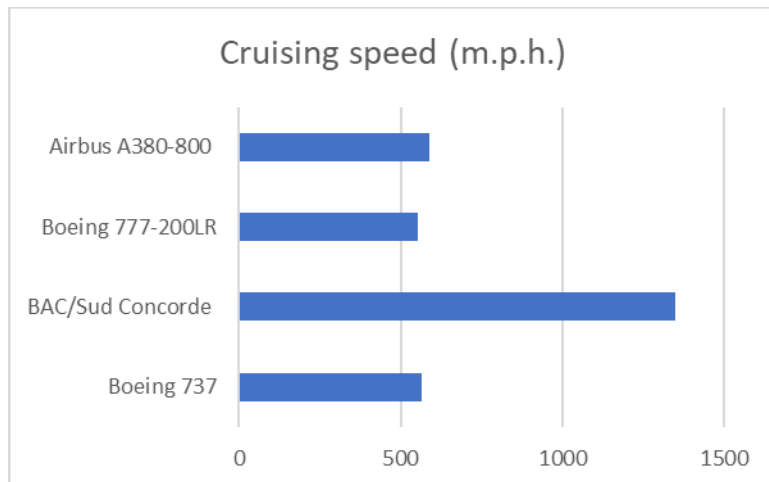
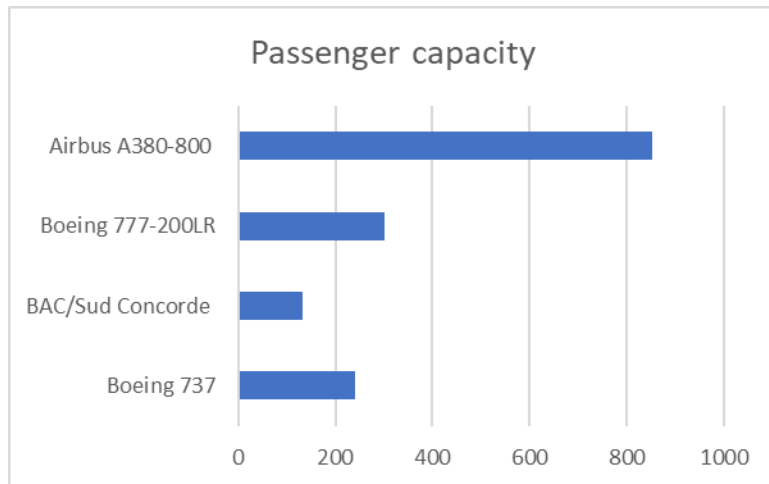
$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area} / 2))^2}$$

- Nonlinear relation to area and defect rate
 - Wafer cost and area are fixed
 - Defect rate determined by manufacturing process
 - Die area determined by architecture and circuit design

Defining Performance

- Which airplane has the best performance?



Response Time and Throughput

- Response time
 - How long it takes to do a task
- Throughput
 - Total work done per unit time
 - e.g., tasks/transactions/... per hour
- How are response time and throughput affected by
 - Replacing the processor with a faster version?
 - Adding more processors?
- We'll focus on response time for now...

Relative Performance

- Define Performance = $1/\text{Execution Time}$
- “X is n time faster than Y”

$$\begin{aligned} \text{Performance}_X / \text{Performance}_Y \\ = \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

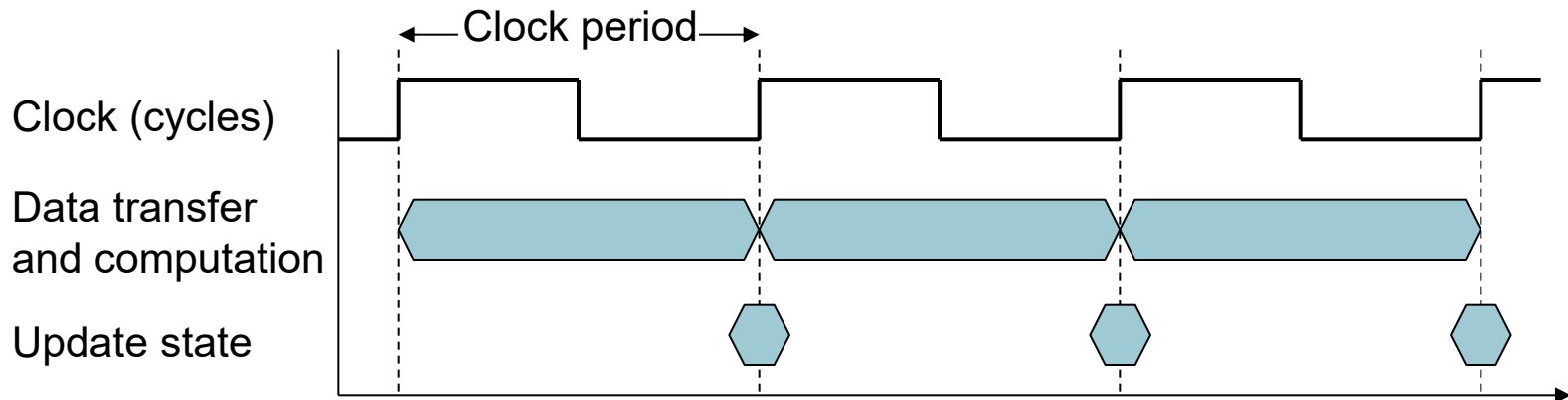
- Example: time taken to run a program
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15\text{s} / 10\text{s} = 1.5$
 - So A is 1.5 times faster than B

Measuring Execution Time

- Elapsed time
 - Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
 - Determines system performance
- CPU time
 - Time spent processing a given job
 - Discounts I/O time, other jobs' shares
 - Comprises user CPU time and system CPU time
 - Different programs are affected differently by CPU and system performance

CPU Clocking

- Operation of digital hardware governed by a constant-rate clock



- Clock period: duration of a clock cycle
 - e.g., $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$
- Clock frequency (rate): cycles per second
 - e.g., $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$

CPU Time

$$\begin{aligned}\text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}}\end{aligned}$$

- Performance improved by
 - Reducing number of clock cycles
 - Increasing clock rate
 - Hardware designer must often trade off clock rate against cycle count

CPU Time Example

- Computer A: 2GHz clock, 10s CPU time
- Designing Computer B
 - Aim for 6s CPU time
 - Can do faster clock, but causes $1.2 \times$ clock cycles
- How fast must Computer B clock be?

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

Instruction Count and CPI

$\text{Clock Cycles} = \text{Instruction Count} \times \text{Cycles per Instruction}$

$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- Instruction Count for a program
 - Determined by program, ISA and compiler
- Average cycles per instruction
 - Determined by CPU hardware
 - If different instructions have different CPI
 - Average CPI affected by instruction mix

CPI Example

- Computer A: Cycle Time = 250ps, CPI = 2.0
- Computer B: Cycle Time = 500ps, CPI = 1.2
- Same ISA
- Which is faster, and by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \leftarrow \text{A is faster...}\end{aligned}$$

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2 \leftarrow \text{...by this much}$$

CPI in More Detail

- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency

CPI Example

- Alternative compiled code sequences using instructions in classes A, B, C

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- Sequence 1: IC = 5

- Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
- Avg. CPI = $10/5 = 2.0$

- Sequence 2: IC = 6

- Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$
- Avg. CPI = $9/6 = 1.5$

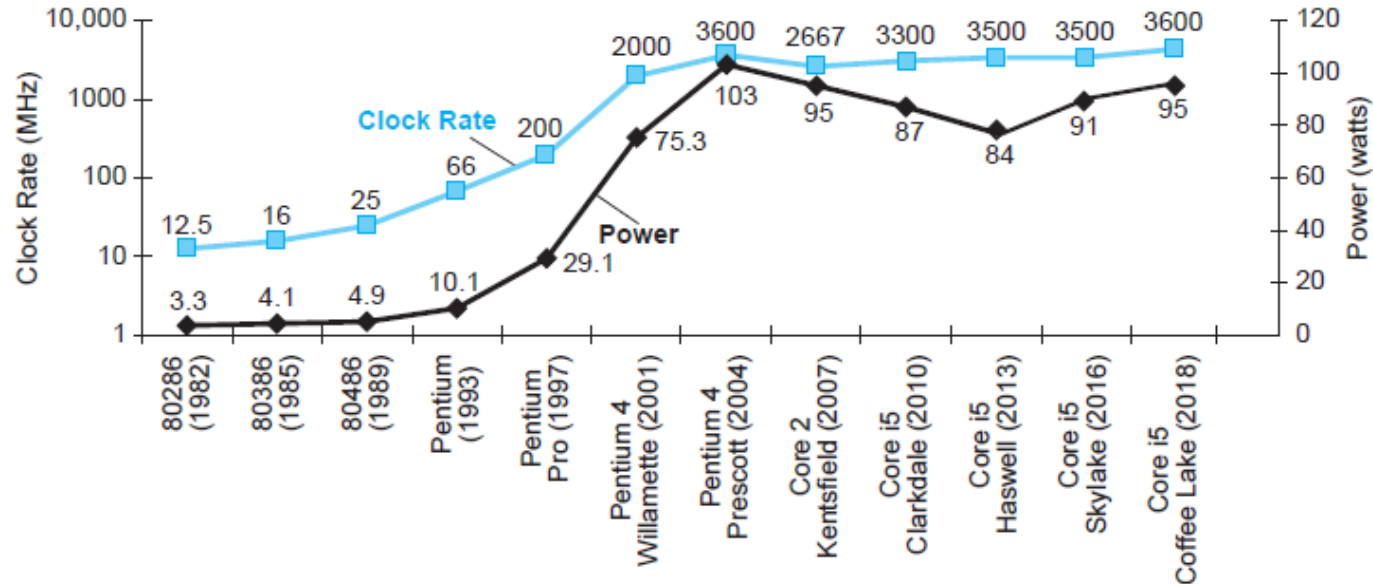
Performance Summary

The BIG Picture

$$\text{CPU Time} = \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}}$$

- Performance depends on
 - Algorithm: affects IC, possibly CPI
 - Programming language: affects IC, CPI
 - Compiler: affects IC, CPI
 - Instruction set architecture: affects IC, CPI, T_c

Power Trends



- In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

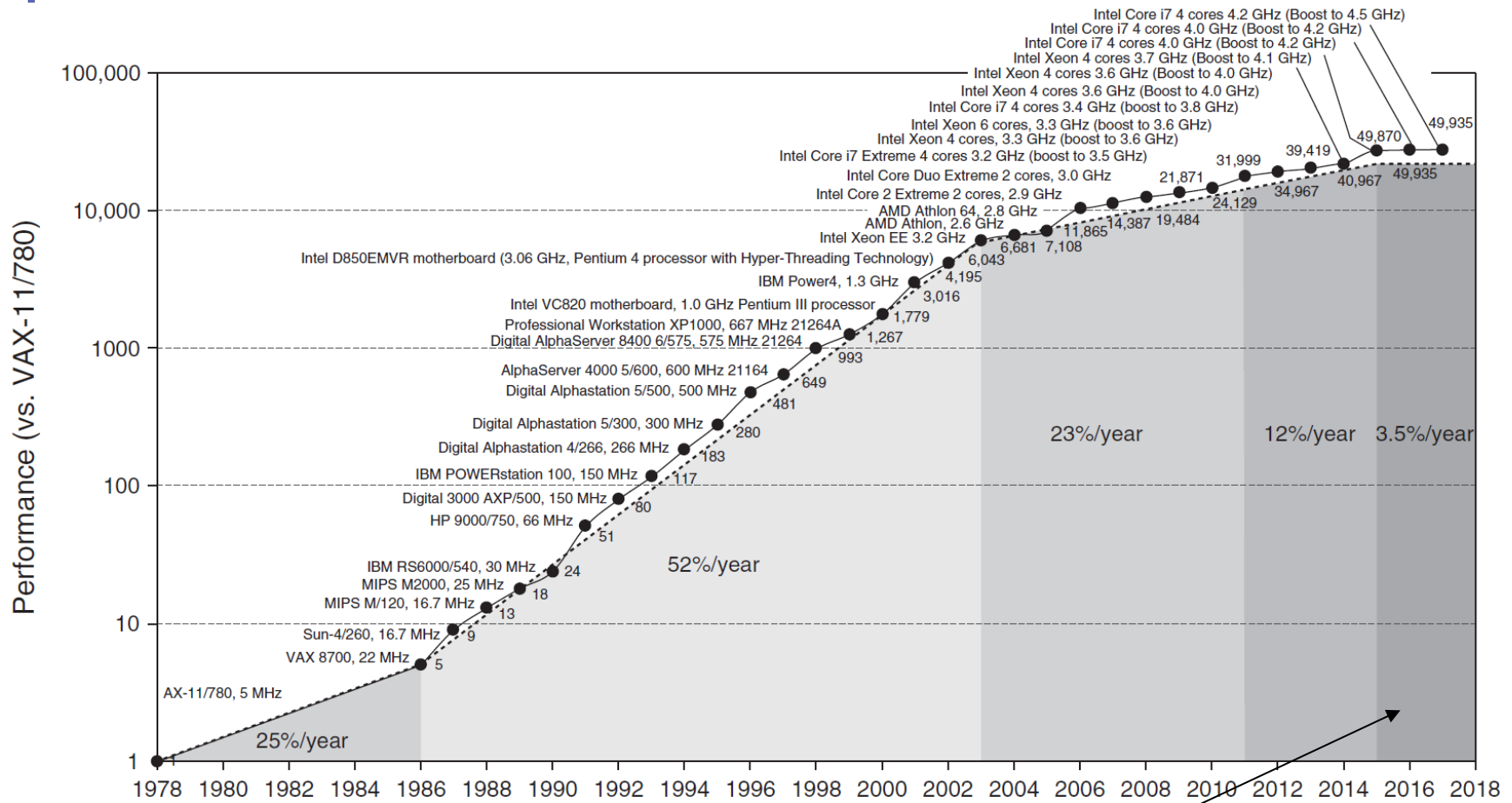
Reducing Power

- Suppose a new CPU has
 - 85% of capacitive load of old CPU
 - 15% voltage and 15% frequency reduction

$$\frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C_{\text{old}} \times 0.85 \times (V_{\text{old}} \times 0.85)^2 \times F_{\text{old}} \times 0.85}{C_{\text{old}} \times V_{\text{old}}^2 \times F_{\text{old}}} = 0.85^4 = 0.52$$

- The power wall
 - We can't reduce voltage further
 - We can't remove more heat
- How else can we improve performance?

Uniprocessor Performance



Constrained by power, instruction-level parallelism, memory latency

Multiprocessors

- Multicore microprocessors
 - More than one processor per chip
- Requires explicitly parallel programming
 - Compare with instruction level parallelism
 - Hardware executes multiple instructions at once
 - Hidden from the programmer
 - Hard to do
 - Programming for performance
 - Load balancing
 - Optimizing communication and synchronization

SPEC CPU Benchmark

- Programs used to measure performance
 - Supposedly typical of actual workload
- Standard Performance Evaluation Corp (SPEC)
 - Develops benchmarks for CPU, I/O, Web, ...
- SPEC CPU2006
 - Elapsed time to execute a selection of programs
 - Negligible I/O, so focuses on CPU performance
 - Normalize relative to reference machine
 - Summarize as geometric mean of performance ratios
 - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i}$$

SPECspeed 2017 Integer benchmarks on a 1.8 GHz Intel Xeon E5-2650L

<i>Description</i>	<i>Name</i>	<i>Instruction Count x 10⁹</i>	<i>CPI</i>	<i>Clock cycle time (seconds x 10⁻⁹)</i>	<i>Execution Time (seconds)</i>	<i>Reference Time (seconds)</i>	<i>SPECratio</i>
Perl interpreter	perlbench	2684	0.42	0.556	627	1774	2.83
GNU C compiler	gcc	2322	0.67	0.556	863	3976	4.61
Route planning	mcf	1786	1.22	0.556	1215	4721	3.89
Discrete Event simulation - computer network	omnetpp	1107	0.82	0.556	507	1630	3.21
XML to HTML conversion via XSLT	xalancbmk	1314	0.75	0.556	549	1417	2.58
Video compression	x264	4488	0.32	0.556	813	1763	2.17
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	2216	0.57	0.556	698	1432	2.05
Artificial Intelligence: Monte Carlo tree search (Go)	leela	2236	0.79	0.556	987	1703	1.73
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	6683	0.46	0.556	1718	2939	1.71
General data compression	xz	8533	1.32	0.556	6290	6182	0.98
Geometric mean							2.36

SPEC Power Benchmark

- Power consumption of server at different workload levels
 - Performance: ssj_ops/sec
 - Power: Watts (Joules/sec)

$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) / \left(\sum_{i=0}^{10} \text{power}_i \right)$$

SPECpower_ssj2008 for Xeon E5-2650L

Target Load %	Performance (ssj_ops)	Average Power (watts)
100%	4,864,136	347
90%	4,389,196	312
80%	3,905,724	278
70%	3,418,737	241
60%	2,925,811	212
50%	2,439,017	183
40%	1,951,394	160
30%	1,461,411	141
20%	974,045	128
10%	485,973	115
0%	0	48
Overall Sum	26,815,444	2,165
$\Sigma \text{ssj_ops} / \Sigma \text{power} =$		12,385

Pitfall: Amdahl's Law

- Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- Example: multiply accounts for 80s/100s
 - How much improvement in multiply performance to get 5× overall?

$$20 = \frac{80}{n} + 20 \quad \text{■ Can't be done!}$$

- Corollary: make the common case fast

Fallacy: Low Power at Idle

- Look back at i7 power benchmark
 - At 100% load: 258W
 - At 50% load: 170W (66%)
 - At 10% load: 121W (47%)
- Google data center
 - Mostly operates at 10% – 50% load
 - At 100% load less than 1% of the time
- Consider designing processors to make power proportional to load

Pitfall: MIPS as a Performance Metric

- MIPS: Millions of Instructions Per Second
 - Doesn't account for
 - Differences in ISAs between computers
 - Differences in complexity between instructions

$$\begin{aligned}\text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6}\end{aligned}$$

- CPI varies between programs on a given CPU

Example on MIPS

- Consider the following performance for a program:

Measurement	Computer A	Computer B
Instruction Count	10 billion	8 billion
Clock Rate	4 GHz	4 GHz
CPI	1.0	1.1

- Which computer has the higher MIPS?
- Which Computer is faster?

$$\text{MIPS}_A = \text{CR}_A / (\text{CPI}_A \times 10^6) = 4\text{GHz} / (1.0 \times 10^6) = 4000 \text{ MIPS}$$

$$\text{MIPS}_B = \text{CR}_B / (\text{CPI}_B \times 10^6) = 4\text{GHz} / (1.1 \times 10^6) = 3636 \text{ MIPS}$$

$$\text{Execution time}_A = (\text{IC}_A \times \text{CPI}_A / \text{CR}_A) \times 10^6 = (10 \times 10^9 \times 1) / 4 \times 10^9 = 2.5 \text{ s}$$

$$\text{Execution time}_B = (\text{IC}_B \times \text{CPI}_B / \text{CR}_B) \times 10^6 = (8 \times 10^9 \times 1.1) / 4 \times 10^9 = 2.2 \text{ s}$$

B is Faster than A

Concluding Remarks

- Cost/performance is improving
 - Due to underlying technology development
- Hierarchical layers of abstraction
 - In both hardware and software
- Instruction set architecture
 - The hardware/software interface
- Execution time: the best performance measure
- Power is a limiting factor
 - Use parallelism to improve performance